Project 3 – EECS 152B

Cameron Peterson-Zopf

Procedure 1: Data Adaptive Beamforming

For the first procedure we assume the desired signal to be at 5 degrees and the two interfering signals to be at -25 degrees and 15 degrees.

Question 1:

In this first question students generate 100,000 samples of data with 16 antennas using the signals in ABF.mat. The code for the generation of said data is displayed below in figure 1.

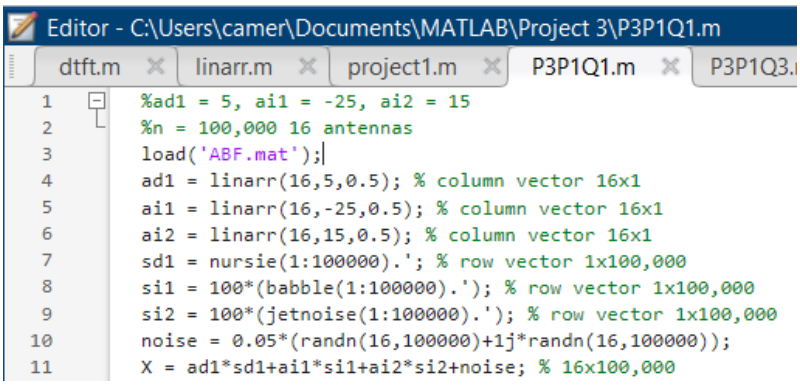

Figure 1 – Code for Generation of Data

Question 2:

Using the first 100 samples of data from question 1, students find the optimal adaptive beamformer and plot its spatial frequency response. The code for the optimal adaptive beamformer is shown in figure 2.

```
%q2 T = 100

A = [ad1 ai1 ai2]; %16x3
c = [1 0 0]'; %3x1
Xn = X(1:16,1:100); %16x100
%gives me time data from first 100 time units
Z = Xn*Xn'; %16x16
Rxx = Z/100; %16x16
Ri = inv(Rxx); %16x16
g = Ri*A*inv(A'*Ri*A)*c;
h = conj(g);
[H,ws]=dtft(h,500);
thax=(180/pi)*asin(ws/pi);
plot(thax,20*log10(abs(H))), xlabel('Degrees'), ...
    ylabel('dB Gain'), title ('dB Gain vs Degrees');
grid
hold
plot([5 5],[-50 10],'k--');
plot([-25 -25],[-50 10],'r--');
plot([15 15],[-50 10],'r--');
sound(real(nursie(1:100000)),22000); %listen to the desired signal
pause(5);
y = h.'*X;
sound(real(X(1,:))); %listen to the first antenna
pause(13);
sound(real(y),22000); %output of beamformer
```

Figure 2 – Code for Adaptive Beamformer w/ T = 100

The plot of the spatial frequency response for the above beamformer is shown in figure 3.

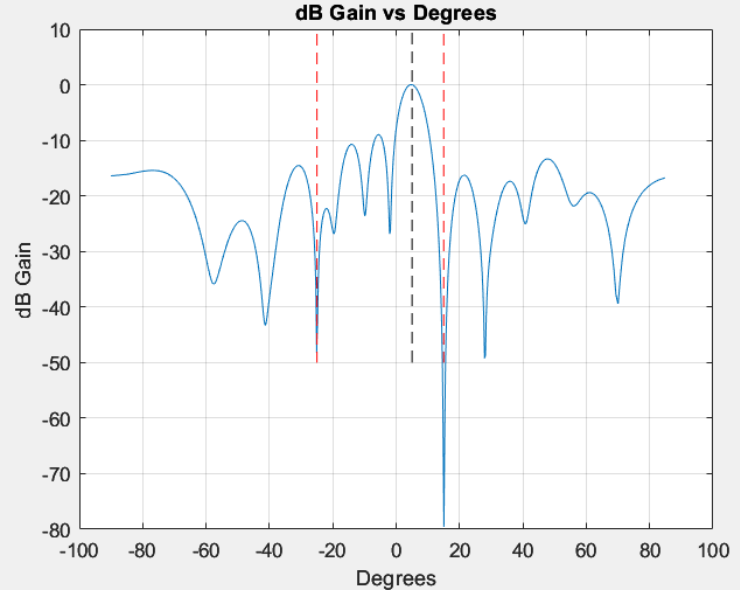

Figure 3 – Adaptive Beamformer w/ T = 100

As seen in figure 3, the beamformer successfully attenuates the signal at -25 degrees and 15 degrees. Furthermore, the gain at the desired signal is unity as desired. Thus, this filter has achieved its goals.

In terms of listening to the output to verify that the beamformer has a clean signal, I first played the desired signal to know what the desired output should be. Then, I listened to the

output of the first antenna, to hear the noisy signal. Lastly, I played the output signal y. As expected, the output of the first antenna was filled with noise. The output signal y matched the original signal, with some slight background noise that was much lower in volume than the desired signal. Therefore, the beamformer was a success.

Question 3:

The next question asks students to repeat the prior experiment, with the exception of moving the interfering angle that was originally at 15 degrees to a location closer to the desired signal at 5 degrees. Thus, I first moved the signal to 10 degrees, and found no issue with the output. I then moved to 7 degrees, or 2 degrees away from the desired signal. At this point, the jet noise begins to increase in loudness; however, I can still make out the desired sound. As I get closer to the 5 degree mark, the jet noise becomes greater and greater. At 5.5 degrees, or 0.5 degrees away from the desired signal, the jet noise is louder than the desired sound, but the desired sound is still recognizable. At around 5.1 degrees, or 0.1 degrees away from the signal, the jet noise becomes so loud, that the original desired signal is no longer recognizable. Figure 4 gives an example of the code employed to create the shifted interfering angle.

Editor - C:\Users\camer\Documents\MATLAB\Project 3\P3P1Q3.m

+6 | linarr.m | project1.m | P3P1Q1.m | P3P1Q3.m | P3

```
%ad1 = 5, ai1 = -25, ai2 = 7
%n = 100,000 16 antennas
load('ABF.mat');
ad1 = linarr(16,5,0.5); % column vector 16x1
ai1 = linarr(16,-25,0.5); % column vector 16x1
ai2 = linarr(16,7,0.5); % column vector 16x1
sd1 = nursie(1:100000).'; % row vector 1x100,000
si1 = 100*(babble(1:100000).'); % row vector 1x100,000
si2 = 100*(jetnoise(1:100000).'); % row vector 1x100,000
noise = 0.05*(randn(16,100000)+1j*randn(16,100000));
X = ad1*sd1+ai1*si1+ai2*si2+noise; % 16x100,000

%q2 T = 100
A = [ad1 ai1 ai2]; %16x3
c = [1 0 0]'; %3x1
Xn = X(1:16,1:100); %16x100
%gives me time data from first 100 time units
Z = Xn*Xn'; %16x16
Rxx = Z/100; %16x16
Ri = inv(Rxx); %16x16
g = Ri*A*inv(A'*Ri*A)*c;
h = conj(g);
sound(real(nursie(1:100000)),22000);
pause(8);
y = h.'*X;
sound(real(y),22000);
```

Figure 4 – Second Interfering Angle at 7 Degrees

As seen above, I simply changed the second interfering angle to different values, closer and closer to 5 degrees, listening to the output sound at each change. The above figure shows this for an interfering angle of 7 degrees.

The interfering angle of 5.1 degrees is the point of failure. Per direction, the amount of antenna is increased to 50. When I added the 50 antennas the result became much better, and I was once again able to recognize my desired signal. The code for this process is shown in figure 5.

Editor - C:\Users\camer\Documents\MATLAB\Project 3\P3P1Q3b.m

+6 | linarr.m | project1.m | P3P1Q1.m | P3P1Q3.m | P3P1Q3b.m

```
%ad1 = 5, ai1 = -25, ai2 = 5.1
%n = 100,000 50 antennas
load('ABF.mat');
ad1 = linarr(50,5,0.5); % column vector 50x1
ai1 = linarr(50,-25,0.5); % column vector 50x1
ai2 = linarr(50,5.1,0.5); % column vector 50x1
sd1 = nursie(1:100000).'; % row vector 1x100,000
si1 = 100*(babble(1:100000).'); % row vector 1x100,000
si2 = 100*(jetnoise(1:100000).'); % row vector 1x100,000
noise = 0.05*(randn(50,100000)+1j*randn(50,100000));
X = ad1*sd1+ai1*si1+ai2*si2+noise; % 50x100,000

%q2 T = 100

A = [ad1 ai1 ai2]; %50x3
c = [1 0 0]'; %3x1
Xn = X(1:50,1:100); %50x100
%gives me time data from first 100 time units
Z = Xn*Xn'; %50x50
Rxx = Z/100; %50x50
Ri = inv(Rxx); %50x50
g = Ri*A*inv(A'*Ri*A)*c;
h = conj(g);
[H,ws]=dtft(h,500);
thax=(180/pi)*asin(ws/pi);
plot(thax,20*log10(abs(H))), xlabel('Degrees'), ...
    ylabel('dB Gain'), title ('dB Gain vs Degrees');
grid
hold
plot([5 5],[-50 10],'k--');
plot([-25 -25],[-50 10],'r--');
plot([5.1 5.1],[-50 10],'r--');
sound(real(nursie(1:100000)),22000);
pause(8);
y = h.'*X;
sound(real(y),22000);
```

Figure 5 – Code for Beamformer with 50 Antennas

The spatial frequency response for figure 5 is plotted in figure 6.

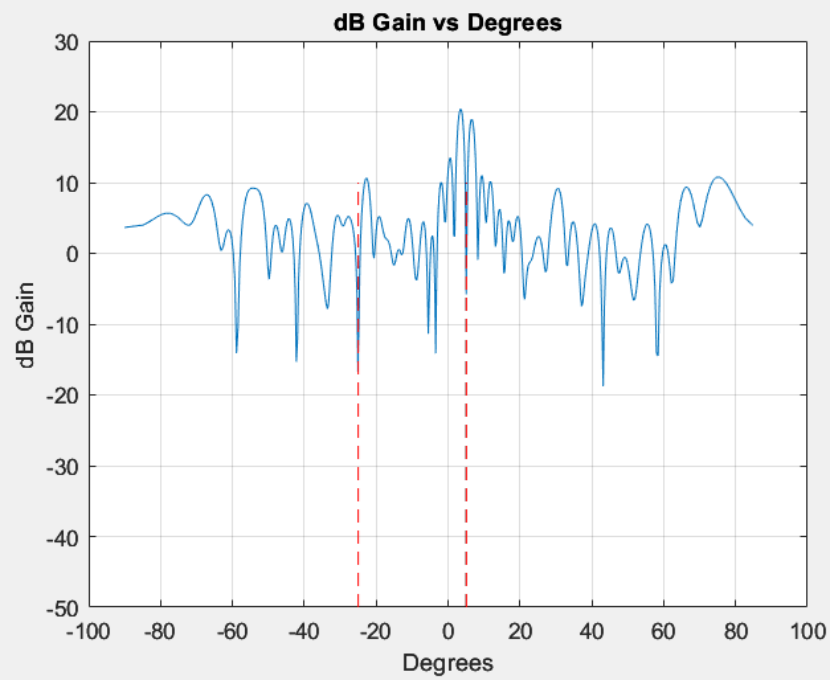

Figure 6 - Spatial Frequency Response of Beamformer with 50 Antennas

Analyzing the graph, we see that the interference signal at -25 degrees is suppressed. At the 5 degree mark, the black line representing the desired signal and the red line representing the interference signals are nearly on top of each other. Thus, figure 7 provides a zoomed in graph of this region.

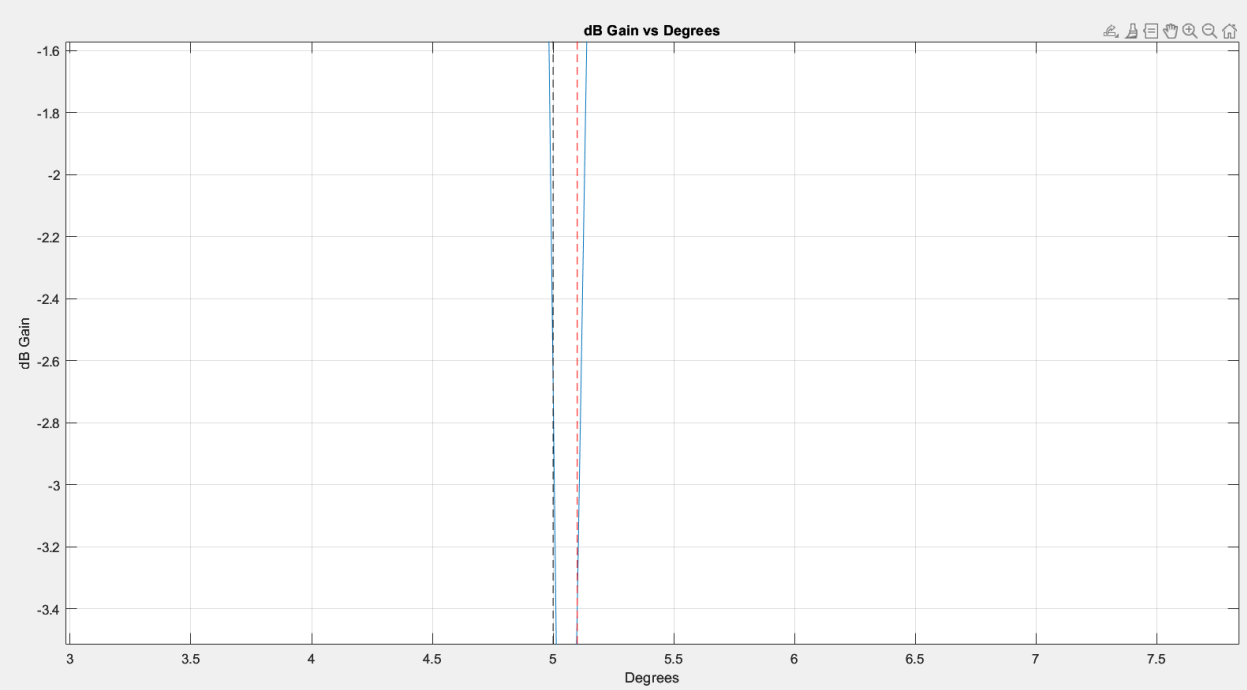

Figure 7 – Zoomed in Spatial Frequency Response

From figure 7, the 5 degree desired signal will experience a gain of -2.5 dB, which is lower than the wanted 0 dB gain. Furthermore, the interference signal at 5.1 degrees experiences a gain of -3.4 dB, which is not that much different than the gain of the desired signal and does not attenuate the unwanted signal very much. However, because we have 50 antennas we are still able to obtain the desired signal, albeit with some interference as heard in the recording.

Procedure 2 – Time and Data Adaptive Beamforming

Question 1:

Utilizing the same scenario as before, but with an additional interference signal beginning at 50,001 samples, or halfway between total 100,000 samples, we add a new interference signal at zero degrees. The code for this process is displayed in figure 8.

Editor - C:\Users\camer\Documents\MATLAB\Project 3\P3P2Q1Retake.m

+6 | P3P1Q3.m | P3P1Q3b.m | GetAng.m | P3P2Q1.m | P3P2Q1Retake.m

```
%ad1 = 5, ai1 = -25, ai2 = 15, ai3 = 0
%n = 100,000 16 antennas
load('ABF.mat');
ad1 = linarr(16,5,0.5); % column vector 16x1
ai1 = linarr(16,-25,0.5); % column vector 16x1
ai2 = linarr(16,15,0.5); % column vector 16x1
ai3 = linarr(16,0,0.5); % column vector 16x1
sd1 = nursie(1:100000).'; % row vector 1x100,000
si1 = 100*(babble(1:100000).'); % row vector 1x100,000
si2 = 100*(jetnoise(1:100000).'); % row vector 1x100,000
JN = 100*(jetnoise(1:50000).'); % row vector 1x50,000
Z = zeros(1,50000); % row vector 1x50,000
si3 = [Z JN]; %halfway through, jet noise begins
noise = 0.05*(randn(16,100000)+1j*randn(16,100000));
X = ad1*sd1+ai1*si1+ai2*si2+ai3*si3+noise; %16x100,000
A = [ad1 ai1 ai2]; %16x3
c = [1 0 0]'; %3x1

e = 0.0005;
Hat0 = zeros(1,100000); %allocate memory for the Hat vector
y = zeros(1,100000); %allocate memory for the output vector
Rxx_np1 = zeros(16,16); %initialize
M = 100;
e1 = 1/M;

for n=1:M
    Z = X(1:16,n)*X(1:16,n)'; %16x16
    Rxx_np1 = (1-e1)*Rxx_np1 + e1*Z;
%this initializes the first value we will use in our calculations below
end
```

```
31
32  for n=M+1:100000
33      Z = X(1:16,n)*X(1:16,n)'; %16x16
34      %sampling at only one time value at a time
35      Rxx_np1 = (1-e)*Rxx_np1 + e*Z;
36          %New value = (1-e)*old value + e*current value
37          %this creates an exponential window filter of past values
38
39      Ri = inv(Rxx_np1); %16x16
40      g = Ri*A*inv(A'*Ri*A)*c;
41      h = conj(g);
42      y(n) = h.'*X(1:16, n); %puts in data into output vector
43      [H,ws]=dtft(h,500);
44      Hat0(n) = H(251); %stores the frequency response at 0 degrees
45  end
46  m=1:100000;
47  plot(m,20*log10(abs(Hat0))), xlabel('Time'), ...
48      ylabel('dB Gain'), title ('dB Gain vs Time');
49      grid
50
51
52  sound(real(nursie(1:100000)),22000);
53  pause(5);
54  sound(real(y),22000);
```

Figure 8 – Code for Beamformer with Additional Interference at 0 Degrees

As seen above, I plot the spatial frequency response at 0 degrees with respect to time. This plot is shown below in figure 9.

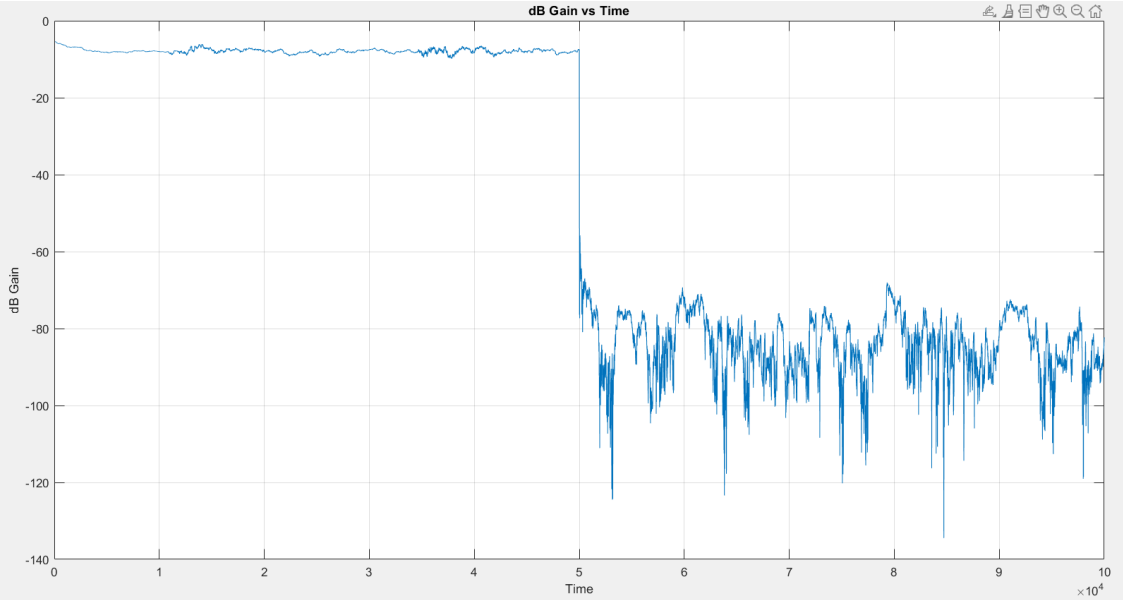

Figure 9 – Plot of Spatial Frequency at 0 degrees vs Time

Zooming in on the 50,001 sample point where the interference kicks in we see the following.

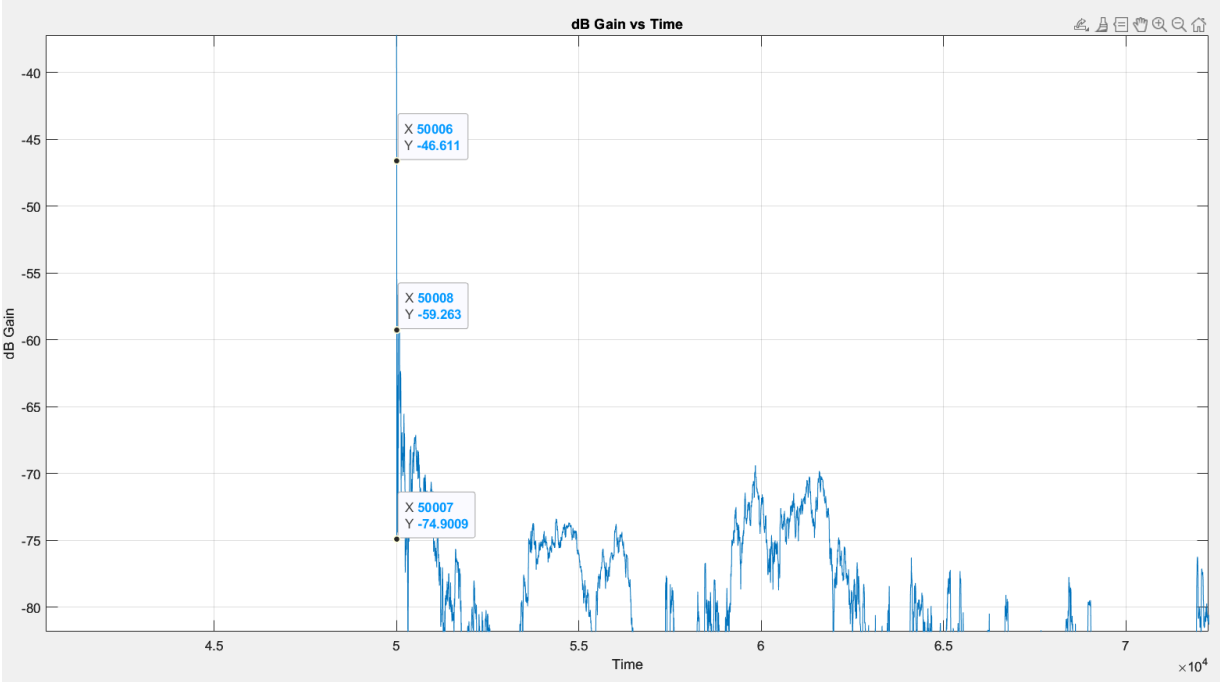

Figure 10 – Zoomed in Plot of Spatial Frequency at 0 degrees vs Time

In Figure 10 we see that the first trough occurs at sample 50,007, which is 6 samples after the interference is introduced. After this point, the response stays at or below -57 dB, which is roughly zero for all intensive purposes. Thus, the beamformer takes 6 samples to adjust to the new interference. Furthermore, upon listening to the output recording, the desired signal can be cleared heard and recognizable.

Notice that I have my epsilon as 0.0005. The choice of this value for epsilon was the best I found given the trade offs of having big and small epsilon values. This analysis of epsilon values will be revisited in question 4.

Question 2:

Question 2 prompts students to write a function that will generate the 100,000 samples of data with time-varying angles for the interference. We assume this variation is linear. The code for the function is shown in figure 11.

Editor - C:\Users\camer\Documents\MATLAB\Project 3\GetAng.m

+6 | P3P1Q3.m | P3P1Q3b.m | GetAng.m | P3P2Q1.m

```
function IntAngle = GetAng(slope, AngSt)
%two input parameters: slope of angle change, starting angle
    n = 100000;
    IntAngle = zeros(1,n); %memory allocation
    for m=1:n
        IntAngle(m) = (m-1)*slope+AngSt; %(1x100000)
    end
end
```

Figure 11 – Function Code for Time-Varying Angles

The function name is “GetAng” and there are two inputs: the slope and the initial angle.

Question 3:

For this question we assume two interfering signals, which begin at -25 degrees and 15 degrees. These two signals will then have their angles vary over time. The first 100 samples are used to initialize the beamformer to cancel the signals at -25 and 15 degrees. Thus, will the beamformer is initializing, I will not move the interfering signals. The variation of angle will begin at the 101 sample. After initializing, the interfering signals will move and the beamformer will adapt to their movement. The code for this process is displayed in figure 12.

Editor - C:\Users\camer\Documents\MATLAB\Project 3\AdptBF.m

+6 | P3P1Q3b.m | GetAng.m | P3P2Q1.m | P3P2Q1Retake.m | AdptBF.m

```
%AngSt1 = -25, AngSt2 = 15, ad1 = 5
%n = 100,000 16 antennas
function OutputS = AdptBF(dang,iang1,iang2,slope,e)
    %First 100 Samples
    load('ABF.mat');
    ad1 = linarr(16,dang,0.5); % column vector 16x1
    ai1 = linarr(16,iang1,0.5); % column vector 16x1
    ai2 = linarr(16,iang2,0.5); % column vector 16x1
    sd1 = nursie(1:100000).'; % row vector 1x100,000
    si1 = 100*(babble(1:100000).'); % row vector 1x100,000
    si2 = 100*(jetnoise(1:100000).'); % row vector 1x100,000
    noise = 0.05*(randn(16,100000)+1j*randn(16,100000));
    X = ad1*sd1+ai1*si1+ai2*si2+noise; %16x100,000

    A = ad1; %16x1
    c = 1; %1x1

    Rxx_np1 = zeros(16,16); %initialize
    M = 100;
    e1 = 1/M;

    for n=1:M
        Z = X(1:16,n)*X(1:16,n)'; %16x16
        Rxx_np1 = (1-e1)*Rxx_np1 + e1*Z;
%this initializes the first value we will use in our calculations below
    end
```

```
27
28  %Update Beamformer:
29      IA1 = GetAng(slope, iang1); %(1x100000)
30      IA2 = GetAng(slope, iang2); %(1x100000)
31
32      Uai1 = zeros(16,100000); %allocate memory
33      Uai2 = zeros(16,100000); %allocate memory
34      UX = zeros(16,100000); %allocate memory
35      adxsd=ad1*sd1;
36      for k=M+1:100000
37          Uai1(1:16,k) = linarr(16,IA1(k-M),0.5);
38          Uai2(1:16,k) = linarr(16,IA2(k-M),0.5);
39          UX(1:16,k) = adxsd(:,k)+Uai1(:,k)*si1(k)+Uai2(:,k)*si2(k)+noise(:,k); %16x10
40      end
41  %now I have gathered all info on int angles for all time
42
43      y = zeros(1,100000); %allocate memory for the output vector
44      for n=M+1:100000
45          Z = UX(1:16,n)*UX(1:16,n)'; %16x16
46  %sampling at only one time value at a time
47          Rxx_np1 = (1-e)*Rxx_np1 + e*Z;
48  %New value = (1-e)*old value + e*current value
49  %this creates an exponential window filter of past values
50          Ri = inv(Rxx_np1); %16x16
51          g = Ri*A*inv(A'*Ri*A)*c;
52          h = conj(g);
53          y(n) = h.'*UX(1:16, n); %puts in data into output vector
54      end
55      OutputS = y;
56  end  %end of function
```

Figure 12 – Code for Time and Data Adaptive Beamformer

Notice that I have written this procedure as a function with 5 parameters: desired signal angle, interference angle #1, interference angle #2, rate of change of angles, and epsilon utilized. I then tested the performance of the beamformer and found the following results for the following scenarios:

1) Holding epsilon constant at 0.0005, and changing the slope from 1/100,000 to 10/100,000 to 20/100,000 to 25/100,000 to 35/100,000 to 100/100,000.
   For this scenario, as the slope increased, the interference became louder, especially near the end of the recording for the slopes of 25/100,000 and 35/100,000. This is because by moving 25 and 35 total degrees, the -25 degree signal is now at 0 degrees and 10 degrees respectively. For the 25/100,000 slope, the desired signal is still recognizable near the end of playtime. However, the 35/100,000 slope is not recognizable near the end of playtime. This is for two reasons: the interference signal beginning at -25 degrees passes over the desired signal, and both of the interference signals change at a fast rate. Lastly, when the slope is increased to 100/100,000, the slope is changing much faster and there is more interference in the output as a result. When the interference signal beginning at -25 degrees passes over the desired signal, the output signal is unrecognizable; but as interference signal gets past the desired signal, we begin to recognize the desired signal once more with some interference.
2) Holding epsilon constant at 0.000000001, and changing the slope from 1/100,000 to 10/100,000 to 20/100,000 to 25/100,000 to 35/100,000 to 100/100,000.

With the epsilon decreased, the beamformer relies more heavily on the previous values. Thus, as the slope increases, the beamformer cannot change quickly enough. For all of slopes tested, the resulting output quality is less than the first scenario for each respective slope, with the slopes of 20/100,000, 25/100,000, 35/100,000, and 100/100,000 being very poor.

At the two scenarios tested above, the faster the interference could move while maintain good cancelation was 35/100,000 for the epsilon of 0.0005 and 20/100,000 for the epsilon of 0.000000001. Figure 13 displays the code for calling the function created in this question.

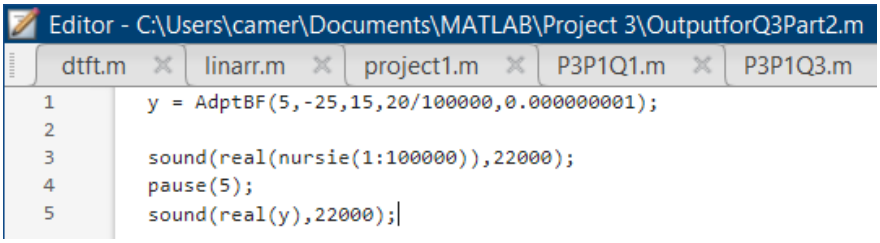

Figure 13 – Generation Output Recordings for Question 3

In the example code above, I am employing a slope of 20/100,000 and a epsilon of 0.000000001.

Question 4:

This question discusses the impact of the various parameters of the adaptive beamformer.

1. Rate of Change of Interfering Signals
   As the rate of change is increased, the adaptive beamformer must change more rapidly to keep up with the increasing rate of change of the interfering signals. Thus, holding epsilon constant, as we increase the rate of change, the sound quality will worsen.
2. Value of Epsilon
   As the value of epsilon decreases the beamformer relies more on the previous values instead of the current one. This leads to greater stabilization and less oscillation in the waveform as additional data points have less effect. However, the downside to this is that the beamformer cannot adjust very quickly. For example, with the first question of this procedure I have my epsilon as 0.0005. This was chosen to minimize the number of samples I needed to be under -55 dB for the remainder of signal. If I increased my epsilon to 0.05, I would reach the mark slightly faster; but then I will have greater oscillations that go above the -55 dB threshold. If I decreased the epsilon to 0.00005, then the oscillations will be greatly suppressed, but then the number of samples needed to reach the mark would have been greater.
3. When the interfering signals move closer to each other, the total interference in that location is very high.

4. When the interfering signals move across the desired signal, the output recording is very poor around when this happens; but is much better at the start and end of the recording when the signals are not near the desired signal.